

on different nodes or channels, which could lead to load imbalances, especially with small query batches. Such imbalances can be mitigated by larger batches, as it is less likely that all queries happen to hit the same node or channel. Additionally, for the case of uneven access frequencies across IVF lists, adjusting their placement based on these frequencies can help achieve better load balancing [9].

5 IMPLEMENTATION

Chameleon is implemented in 11K lines of code, including 3K lines of Vitis HLS C/C++ for the ChamVS near-memory accelerator, 1.4K lines of C++ for the CPU coordinator, 3.5K lines of Python for ChamLM, and 3.2K lines of Python for various evaluations. Referring to existing RALM research [41, 42], we build ChamLM on Fairseq [60], a PyTorch-based LLM toolkit. ChamLM extends Fairseq to support multi-GPU inference, initiating retrieval requests, integrating the retrieved tokens into generation processes, and network communication between the retrieval engines and GPU processes. ChamVS.idx uses Faiss [40] for index scanning on GPUs or CPUs. ChamVS.mem integrates an FPGA TCP/IP stack [25]. The CPU coordinator process for query broadcasting and result aggregation is implemented in C++ using the socket library. The simple messages in RALMs allow us to avoid higher-level abstractions like RPCs, minimizing performance overhead.

6 EVALUATION

We evaluate Chameleon to answer the following questions:

- How much performance and energy benefits can ChamVS attain in large-scale vector search? § 6.2
- How does Chameleon perform across different RALMs by introducing heterogeneous accelerators? § 6.3
- Is accelerator disaggregation necessary? § 6.3

6.1 Experimental Setup

LLMs. We evaluate RALM models of similar sizes to those in existing RALM research [7, 31, 51, 72, 86], up to several billions of parameters. We evaluate both smaller (S) and larger (L) decoder-only (Dec) and encoder-decoder (EncDec) models. Table 1 summarizes the four RALMs for evaluation, including input dimensionalities, numbers of layers and attention heads, model sizes, retrieval intervals, and neighbor numbers. For EncDec models, we follow [7] to use a two-layer shallow encoder and a deeper decoder, and set different retrieval intervals. For all the models, we use a vocabulary size of 50K and let them generate 512 tokens per sequence.

Vector datasets. Table 2 summarizes the four evaluated vector datasets. The SIFT and Deep datasets are popular benchmarks for billion-scale ANN. Due to the lack of available datasets for RALM, we create two synthetic datasets by replicating each SIFT vector to the models’ dimensionalities (512 and 1024). As a common practice, we set *nlist*, the number of clusters in the IVF index, to approximately the square root of the number of dataset vectors (*nlist*=32K). We set *nprobe* as 32 to scan 0.1% of database vectors per query, for which high recall can be achieved on both real-world datasets (93% on Deep and 94% on SIFT for 100 nearest neighbors). We quantize the SIFT and Deep datasets to 16-byte PQ codes, while the two synthetic datasets adopt 32 and 64-byte PQ codes, respectively.

Table 1: Various RALM configurations in the evaluation.

	Dim.	Layers	Heads	Param.	Interval	K
Dec-S	512	24	8	101M	1	100
Dec-L	1024	96	16	1259M	1	100
EncDec-S	512	2,24	8	158M	8/64/512	10
EncDec-L	1024	2,96	16	1738M	8/64/512	10

Table 2: The vector datasets used in the evaluation.

	Deep	SIFT	SYN-512	SYN-1024
#vec	1E+9	1E+9	1E+9	1E+9
<i>m/D</i>	16/96	16/128	32/512	64/1,024
<i>nprobe/nlist</i>	32/32K	32/32K	32/32K	32/32K
Raw vectors (GB)	384	512	2,048	4,096
PQ and vec IDs (GB)	24	24	40	72

Software. For vector search, we use *Faiss* [1] developed by Meta, known for its optimized PQ implementations for both CPUs and GPUs. Due to its vector-only nature, *Faiss*’s ANN search performance surpasses vector data management systems that support additional relational data functionalities [62]. For LLM inference, we extend Fairseq [60] to support RALMs as introduced in §5.

Hardware. We instantiate the ChamVS near-memory accelerator on AMD Alveo U250 FPGAs (16 nm) equipped with 64 GB of DDR4 memory (4 channels x 16 GB) and set the accelerator frequency to 140 MHz. For a fair comparison, each ChamVS memory node is compared to a CPU-based vector search system with equivalent memory capacity (64 GB) and an 8-core AMD EPYC 7313 processor (7 nm) with a base frequency of 3.0 GHz. We evaluate NVIDIA RTX 3090 GPUs (8nm) with 24 GB GDDR6X memory.

6.2 Large-Scale Vector Search on ChamVS

Search performance. We compare ChamVS with baseline systems using four hardware setups. PQ codes can be processed on CPU/FPGA while the IVF index can be scanned on CPU/GPU, leading to four hardware configurations: CPU, CPU-GPU, FPGA-CPU, and FPGA-GPU. To report the best baseline performance, the CPU and CPU-GPU systems are monolithic, while the FPGA-CPU and FPGA-GPU systems are disaggregated over the network. Figure 8 compares the latency distributions of the four solutions. Each white dot in the violin plots denotes a median latency. The number of CPU cores and the number of accelerators used are listed in the plot legends. We make two primary observations from the experiments:

Firstly, the near-memory accelerator in ChamVS significantly lowers vector search latency. Across different datasets and batch sizes (Figure 8), the FPGA-CPU solution achieves 1.36~6.13× speedup compared to the CPU baseline, and the FPGA-GPU solution shows even higher speedup (2.25~23.72×). This is because the ChamVS near memory accelerator can (a) decode PQ codes in parallel, (b) pipeline the decoding, distance calculation, and K-selection, such that each quantized vector can be processed by the pipeline rapidly.

Secondly, scanning the IVF index on GPU allows further latency improvements compared to the FPGA-CPU solution. As shown in Figure 8, the FPGA-GPU approach achieves 1.04~3.87× speedup compared to the FPGA-CPU solution. This is because the IVF index scan procedure can easily leverage the massively parallelism and the high memory bandwidth of GPUs. In contrast, the hybrid CPU-GPU solution shows little or even negative improvements compared

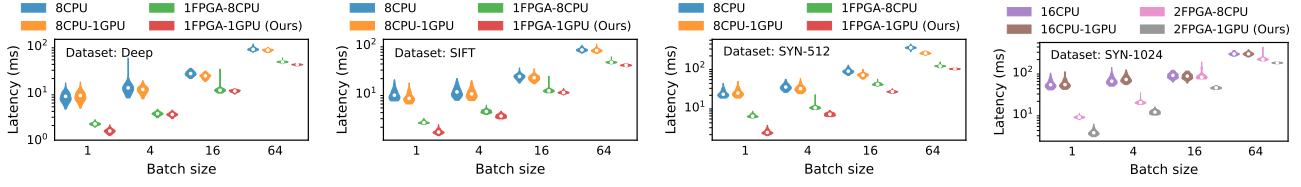


Figure 8: ChamVS achieves significantly lower vector search latency than CPUs and GPUs.

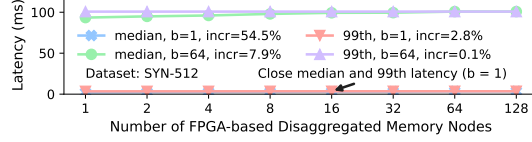


Figure 9: The performance scalability of ChamVS.

Table 3: Average energy consumption per query (in mJ) on ChamVS and CPUs using various batch sizes (1~16).

	CPU			ChamVS (FPGA + GPU)		
	b=1	b=4	b=16	b=1	b=4	b=16
SIFT	950.3	434.0	143.3	53.6	28.2	21.5
Deep	929.5	412.9	141.9	52.3	26.9	20.5
SYN-512	1734.9	957.8	372.5	95.6	55.0	41.1
SYN-1024	4459.9	2315.0	918.5	170.1	107.8	85.2

to the CPU-only solution (0.91~1.42 $\times$), because the search performance is limited by the slow PQ code scan process on CPU.

Scalability. We extrapolate query latency beyond the limited number of accelerators available in our evaluation. Considering the one-GPU and N -FPGA setup, we estimate the latency distribution by summing up accelerator and network latencies. Each query latency number is the maximum of N randomly sampled latency numbers from the 1-FPGA setup. For network latency, we assume a 100 Gbps bandwidth for the CPU server and apply the LogGP model [5, 13], which assumes a tree topology for broadcast and reduce communications, setting the latency between two endpoints as 10.0 μ s (a conservative number compared to 6.0 μ s reported in [26, 27]). Figure 9 presents the median and the 99th percentile latencies for different batch sizes on the SYN-512 dataset. The tail latencies remain almost identical to those in the one-node setup due to the negligible network latency compared to the query. As for the median latencies, there is only a 7.9% increase for a batch size of 64, while for the case without batching, the latency increases by 54.5% as the accelerator latency is determined by the slowest one.

Energy consumption. ChamVS achieves 5.8~26.2 $\times$ energy efficiency compared to the CPU. Table 3 summarizes the average energy consumption to serve a single query across different systems. We measure CPU, GPU, and FPGA energy consumption using Running Average Power Limit (RAPL) and NVIDIA System Management Interface, and Vivado, respectively. For ChamVS, we report the energy per query by measuring the power consumption times latency for scanning index on GPU and scanning PQ codes on FPGAs, respectively, and summing the two parts up.

Recall. ChamVS, with approximate hierarchical priority queues (AHPQ), delivers results nearly identical to those of the software. Table 4 shows the recall given various AHPQ lengths (8~32) when searching for the $K = 100$ nearest neighbors. Here, R1@100 indicates the percentage of queries where the top nearest neighbor is within the results, while R@100 represents the percentage of

Table 4: Recall of ChamVS using approximate queues.

	CPU (len=100)	AHPQ (len=8)	AHPQ (len=16)	AHPQ (len=32)
R1@100 (Deep)	92.88%	92.85%	92.84%	92.84%
R@100 (Deep)	45.54%	45.49%	45.49%	45.48%
R1@100 (SIFT)	94.21%	94.20%	94.21%	94.21%
R@100 (SIFT)	48.68%	48.66%	48.67%	48.67%

overlap between the true 100 nearest neighbors and the 100 results returned. Compared to software, AHPQ only decreases recall by up to 0.06%. Interestingly, on the Deep dataset, reducing the queue lengths to eight does not necessarily result in lower recall than using a length of 32. This is likely due to the nature of PQ approximation — a higher distance indicated by PQ does not always mean that the original vector is actually farther from the query.

6.3 End-to-end RALM Inference on Chameleon

We evaluate RALM inference performance on Chameleon with different models and retrieval intervals, using the SYN-512 and SYN-1024 datasets for the smaller and larger models, respectively.

RALM performance. We evaluate system performance when generating a 512-token sequence using a single GPU for LLM inference. For the latency evaluation, we disable batching, while the throughput evaluation uses the maximum allowed batch sizes given the GPU’s memory capacity (64 for Dec-S and EncDec-S; 8 for Dec-L and EncDec-L). For vector search in RALM, we use the FPGA-GPU solution for ChamVS and the CPU-only solution as the baseline, as CPU-GPU vector search can be even slower using small batches.

Chameleon significantly outperforms the CPU-GPU baseline system in latency for inference steps involving vector search. Figure 10 visualizes the RALM inference latency of Chameleon and the baseline system (CPU-GPU) for the first 128 generated tokens. Inference latency is represented by the grey dots, while retrieval latency accounts for the remaining portion of the end-to-end latency. The time spent on coordinator and index scanning is not marked in the figure, as their latencies of hundreds of microseconds are negligible compared to up to tens of milliseconds for inference and retrieval. Figure 10 shows that ChamVS significantly reduces the latency at the token generation steps requiring retrieval, as the retrieval latency of Chameleon is almost negligible compared to the inference latency executed on GPUs. Specifically, the speedup provided by Chameleon at retrieval-based inference steps (retrieval + inference) ranges from 1.94~4.11 $\times$, 1.71~3.02 $\times$, 1.76~3.41 $\times$, and 1.29~2.13 $\times$ for Dec-S, EncDec-S, Dec-L, and EncDec-L, respectively.

Chameleon achieves up to 3.18 $\times$ throughput compared to the CPU-GPU baseline. Figure 11 shows that the lower the retrieval interval, the more throughput advantage Chameleon offers, with the speedup being 3.18 $\times$ and 2.34 $\times$ for Dec-S and Dec-L that require retrieval per token generation (interval=1). Chameleon attains greater speedup in batched inference than single-sequence inference (as in latency

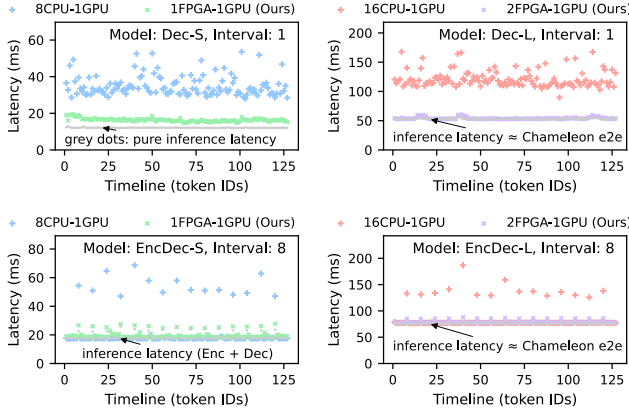


Figure 10: Latency of RALM inference given different LLM configurations and retrieval intervals.

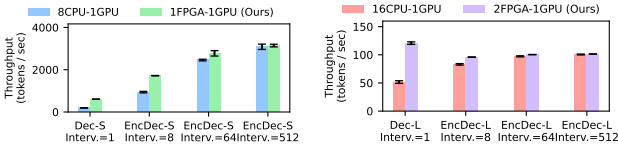


Figure 11: Throughput of RALM inference given different LLM configurations and retrieval intervals.

experiments), because, as the batch size grows, the latency increase for LLM inference is not as significant as that of vector search, due to the many-core parallelism that GPUs offer.

The need for resource disaggregation. Accelerator disaggregation allows Chameleon to adjust the ratio between the two types of accelerators across RALM configurations. We model the overall system throughput, measured by generated tokens per second, across various accelerator ratios using a total of 1,000 accelerators, assuming the cost for an inference accelerator and a retrieval accelerator is equivalent. Given retrieval interval i , batch size b , number of inference and retrieval accelerators N_I and N_R , latency per batch for inference and retrieval $L_I(b)$ and $L_R(b)$, the system throughput is determined by the minimum of the inference and retrieval throughput: $Th_{system} = \min(Th_I, Th_R)$, where $Th_I = \frac{i \cdot b \cdot N_I}{i \cdot L_I(b) + L_R(b)}$ and $Th_R = \frac{i \cdot b \cdot N_R}{L_R(b)}$. Figure 12 shows that the optimal ratio of accelerators to achieve the highest throughput varies significantly, ranging from 53.7%~99.0% across RALMs.

The disaggregated design, using the optimal accelerator ratio, consistently outperforms the monolithic ones with fixed ratios, as shown in Figure 13. Given the impracticality of adjusting the ratio for each RALM in a monolithic design, the performance of a monolithic design can only match that of Chameleon on a limited set of RALMs.

7 RELATED WORK

To our knowledge, Chameleon represents the first endeavor to improve RALM inference performance by using heterogeneous accelerator systems. We now introduce related research topics below.

ANN search. Researchers have developed various ANN search algorithms [14, 19, 30, 36, 54, 61, 75, 78, 84, 92, 93] and data management systems [22, 58, 62, 77, 79, 85]. Apart from PQ-based vector search, graph-based searching algorithms [17, 18, 53, 55, 56, 65, 81,

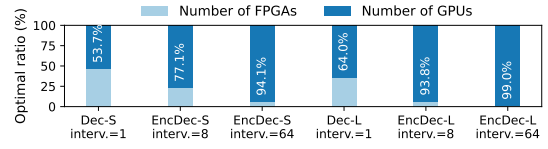


Figure 12: Disaggregation is essential as the optimal accelerator ratio varies significantly across RALM configurations.

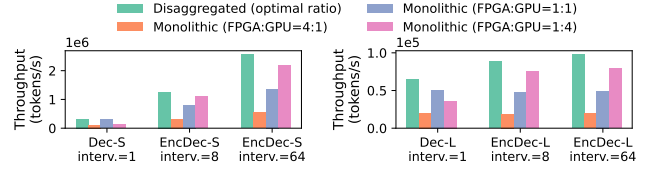


Figure 13: The disaggregated design consistently outperforms the monolithic ones using fixed accelerator ratios.

91, 94] are popular as they can achieve high recall and low latency. Locality-sensitive hashing (LSH) [15, 21] offers theoretical guarantees in ANN search but empirically does not perform as well as PQ and graph-based algorithms.

Vector search on modern hardware. Faiss is the most popular GPU-accelerated ANN search library so far [40], and there are several academic GPU implementations [10, 11, 52, 80]. Lee et al. [45] study ASIC designs for IVF-PQ, and a couple of works [38, 89] implement IVF-PQ on an FPGA, but their designs are constrained by either the limited HBM capacity or the slow CPU-FPGA interconnect. In contrast, Chameleon disaggregates IVF-PQ, with the index on GPUs and PQ codes on FPGA-based memory nodes, and employs the innovative hardware priority queue design to achieve high performance with little hardware resources. While graph-based vector search accelerators can achieve low latency [37, 88], the memory consumption is high, requiring up to one TB of memory for only one billion SIFT vectors, in contrast to 24 GB in our case. Apart from accelerators, researchers also study memory and storage for vector search. One can leverage non-volatile memory [71] and CXL [33] to scale up graph-based ANN, while on-disk ANN has to be more careful with I/O cost [9, 34, 47]. Hu et al. [28] further push down distance evaluation into NAND flash to reduce data movement.

8 CONCLUSION AND OUTLOOK

We present Chameleon, a heterogeneous and disaggregated accelerator system for efficient RALM inference. Given the rapidly evolving algorithms, software, and hardware related to RALMs, Chameleon can be potentially upgraded in the following ways. For LLM inference, ChamLM could be enhanced by supporting low precision [2], continuous batching [87], paged-attention [44], and disaggregated prompt computation and token generation [63]. While currently supporting PQ, ChamVS could potentially be replaced by graph-based ANN accelerators [64, 88]. ChamVS could also be extended to support index updates [83] and relational features [90].

ACKNOWLEDGMENTS

We thank AMD for their generous donation of the Heterogeneous Accelerated Compute Clusters (HACC) at ETH Zurich (<https://systems.ethz.ch/research/data-processing-on-modern-hardware/hacc.html>), on which the experiments were conducted.